K. SHALINI

192472213

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO5: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr G .A. SENTHIL

QUESTIONS: 10 **Total Marks: 50**

Annexure A

Scenario-Based Assessment **DURATION: 1.5 Hrs**

1. A mobile app company wants to improve user engagement by showing personalized notifications at the right time. The company plans to use Reinforcement Learning to learn user behavior patterns. Report how the RL framework can be applied by defining the possible states, actions, and reward functions for this scenario. **(5 Marks)**

2. A warehouse robot uses Reinforcement Learning to find the shortest path for delivering packages while avoiding obstacles. Sometimes the robot chooses longer paths due to insufficient exploration during training. Solve how the exploration vs exploitation trade-off affects the robot's navigation performance and suggest some improvements. **(5 Marks)**

3. A hospital is considering using Reinforcement Learning to optimize patient appointment scheduling in order to reduce waiting time and improve resource utilization. Demonstrate the suitability of RL for this scenario and discuss possible challenges such as delayed rewards and ethical considerations. **(5 Marks)**

4. A smart home system aims to reduce electricity consumption by automatically controlling lights, fans, and air conditioners based on user presence and weather conditions. Sketch a Reinforcement Learning model for this scenario by defining state space, action space, reward structure, and suitable learning algorithm.

(5 Marks)

5. An online gaming platform uses Reinforcement Learning to match players of similar skill levels to maintain engagement. Report how reward design can influence matchmaking quality with reward definition that might lead to biased or inefficient matching results. **(5 Marks)**

6. A financial trading system applies Reinforcement Learning to decide buy/sell actions based on market signals. Analyze how states, actions, and rewards can be defined in this context and evaluate the risks of delayed rewards in volatile markets.

(5 Marks)

7. A drone delivery company uses RL to optimize delivery routes while avoiding restricted airspace. Construct a suitable RL framework by defining states, actions, and rewards, and justify how penalties can ensure safe navigation. **(5 Marks)**

8. A university applies RL to personalize online learning paths for students. Examine how reward design can influence student engagement and defend the ethical considerations of adapting learning content dynamically. **(5 Marks)**

9. A traffic management authority deploys RL to control traffic lights at busy intersections. Illustrate how constraints such as emergency vehicle priority and pedestrian safety can be incorporated into the RL model and interpret the impact on congestion reduction. **(5 Marks)**

10. A manufacturing plant uses RL to schedule machines under resource limitations. Differentiate between traditional optimization methods and constrained RL approaches, and assess which method provides better adaptability in dynamic production environments. **(5 Marks)**

Code of Conduct:

I K. SHALINI [192472213] certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. Personalized Notifications Using Reinforcement Learning

1. Understanding of the Scenario

A mobile app company wants to **increase user engagement** by sending personalized notifications at the **most appropriate time**.

Reinforcement Learning can learn from the user's previous interactions and gradually identify **which notification should be sent, when it should be sent, and when it should not be sent**.

The RL framework can be represented as:

Agent → Notification Recommendation System

Environment → Mobile App + User

State → Current user behavior/context

Action → Notification decision

Reward → User's response to the notification

2. Definition of States

A **state** represents the current situation of the user before the system decides what to do.

Possible state variables are:

State Information	Example
Time of day	Morning, afternoon, evening
Day	Weekday/weekend
User activity	Active, inactive
Recent app usage	Used 10 minutes ago
Previous notification response	Opened / ignored
Notification frequency	Number of notifications received today
User preference	Prefers evening notifications
Context	Working, travelling, etc.

For example:

State S_1: User is active, it is 7 PM, user frequently opens the app at this time, and the last notification was opened.

This information helps the agent decide the next action.

3. Definition of Actions

An **action** is the decision taken by the RL agent.

Possible actions are:

1. **Send notification now**
2. **Send notification later**
3. **Do not send notification**
4. **Send a personalized notification**
5. **Choose a particular notification type**

For example:

If the user is highly active at 7 PM, the agent may choose **"Send notification now."**

4. Reward Function

The **reward** tells the RL agent whether its action was good or bad.

A suitable reward function could be:

User Response	**Reward**
User opens notification and uses app	**+10**
User clicks notification	**+5**
User opens app later	**+3**
User ignores notification	**0**
User dismisses notification	**-3**
User disables notifications/uninstalls app	**-10**

A simple reward function can be written as:

The agent tries to **maximize the total long-term reward**, rather than only maximizing immediate clicks.

5. How the RL Framework Works

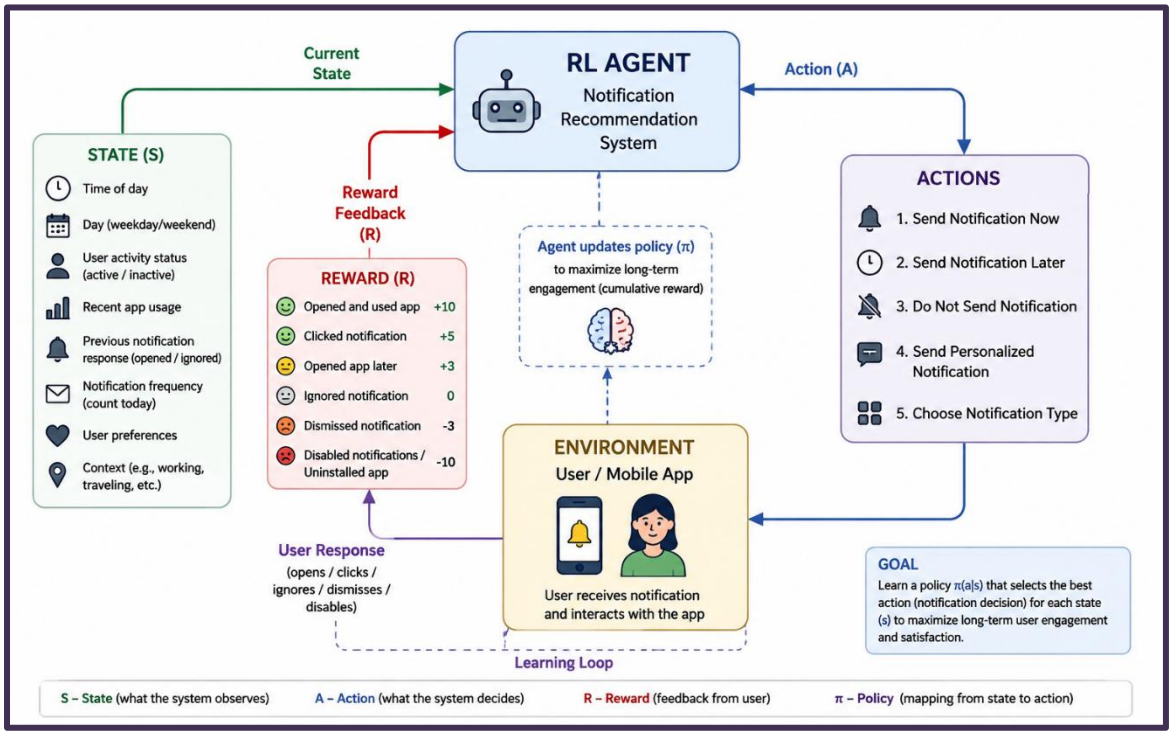
The process is:

User State → Agent selects notification action → User responds → Reward → Agent learns → Improved future decision

For example:

1. The system observes that a user usually uses the app at **8 PM**.
2. The agent sends a notification at 8 PM.
3. The user opens the notification and uses the app.
4. The agent receives a **positive reward**.
5. The system learns that this time and notification type are effective.
6. In the future, it is more likely to send similar notifications at an appropriate time.

If the user repeatedly ignores notifications sent at 10 AM, the system receives little or negative reward and gradually learns to **avoid that timing**.



6. Decision Making

A **policy-based RL approach** can be used where the policy learns:

where:

- = current user state
- = notification action
- = probability of selecting action in state

The policy is gradually improved based on user responses and rewards.

Recommended decision: The agent should not simply send notifications whenever possible. It should learn the **best notification action and timing for each user state** while avoiding excessive notifications.

7. Final RL Model

RL Component	In This Scenario
Agent	Notification recommendation system
Environment	Mobile app and user
State	User activity, time, preferences, previous responses, etc.

Action	Send now, delay, don't send, choose notification type
Reward	Engagement, clicks, ignoring, dismissal, disabling notifications
Policy	Determines the best notification action for each state
Goal	Maximize long-term user engagement

Conclusion

Thus, Reinforcement Learning can continuously learn from **user behavior and feedback** to personalize notification timing and content. By defining appropriate **states, actions, and rewards**, the system can learn a policy that increases engagement while reducing unwanted notifications.

Q2. Exploration vs. Exploitation in Warehouse Robot Navigation

1. Understanding of the Scenario

A warehouse robot uses **Reinforcement Learning (RL)** to find the shortest path for delivering packages while avoiding obstacles.

The robot must decide between:

- **Exploration:** Try new paths to discover potentially shorter or safer routes.
- **Exploitation:** Use a path that it already knows gives a good reward.

The problem is that the robot has **insufficient exploration**, so it repeatedly chooses familiar paths even when a shorter path may exist.

RL Components

Component	Warehouse Scenario
Agent	Warehouse robot
Environment	Warehouse
State	Robot position, obstacles, destination, nearby paths
Actions	Move forward, backward, left, right
Reward	Positive for reaching destination; negative for long paths/collisions
Goal	Find a short, safe and efficient path

2. Key Issue: Exploration vs. Exploitation

Exploration

The robot tries **new or less-visited paths**.

Advantage:

It may discover a shorter or safer route.

Disadvantage:

It may temporarily take longer routes or make mistakes.

Exploitation

The robot chooses the **best-known path** based on its previous learning.

Advantage:

It can deliver packages quickly using a known good route.

Disadvantage:

If exploration is insufficient, the robot may get stuck using a **suboptimal longer path**.

Effect on Performance

In this scenario:

Too little exploration → robot does not discover better paths → repeatedly follows longer routes → lower navigation efficiency.

However, excessive exploration can also reduce performance because the robot may keep trying unnecessary paths instead of using a known efficient route.

Therefore, the robot needs a **balance between exploration and exploitation**.

3. Example

Suppose the robot has discovered:

Path A → 15 steps

Path B → 10 steps

Path C → Unknown

If the robot always exploits its currently known best path, it may choose **Path B**.

But suppose **Path C actually requires only 7 steps**.

If the robot never explores Path C, it will never discover this better solution.

Therefore:

Insufficient exploration → Path B continues to be selected → optimal Path C remains undiscovered.

4. Suitable Improvement: ε-Greedy Strategy

A simple solution is to use the **ε-greedy strategy**.

The robot chooses:

For example, if:

then:

- **20% probability:** Explore a new action/path.

- **80% probability:** Exploit the best-known path.

This allows the robot to discover new routes while still using good routes most of the time.

5. ε Decay

A better approach is to **start with high exploration and gradually reduce it**.

For example:

Training begins

↓

High ε

↓

More exploration

↓

Discover different paths

↓

Learn good routes

↓

Reduce ε gradually

↓

More exploitation

↓

Use shortest learned path

Example:

Training Stage	**ε**	**Behavior**
Early	1.0	Mostly explore
Middle	0.5	Balanced exploration
Later	0.1	Mostly exploit
Final	0.01	Mostly use best path

This is called an **ε-decay strategy**.

6. Other Improvements

A. Reward Shaping

Give appropriate rewards to encourage efficient navigation.

For example:

- **+100** → Reaches destination
- **−1** → Each movement step
- **−20** → Collision with obstacle
- **−5** → Taking an unnecessarily long route

The step penalty encourages the robot to find shorter paths.

B. Experience Replay

Store previous experiences:

in a replay memory and train using randomly selected experiences.

This helps the agent learn from both **successful and unsuccessful paths**.

C. Prioritized Experience Replay

Experiences that contain important information, such as successful discovery of a much shorter path, can be sampled more frequently.

D. Reward for New-State Exploration

The robot can receive a small additional reward for visiting useful unexplored areas, encouraging it to discover alternative routes.

7. Recommended Decision

Best solution: Use **ε-greedy exploration with ε-decay**, combined with an appropriate reward function.

Why?

1. High exploration at the beginning helps discover different warehouse routes.
2. Gradual reduction of ε allows the robot to exploit learned knowledge.
3. Step penalties encourage shorter paths.
4. Collision penalties encourage safe navigation.
5. The robot eventually converges toward an efficient route.

8. Diagram — Required

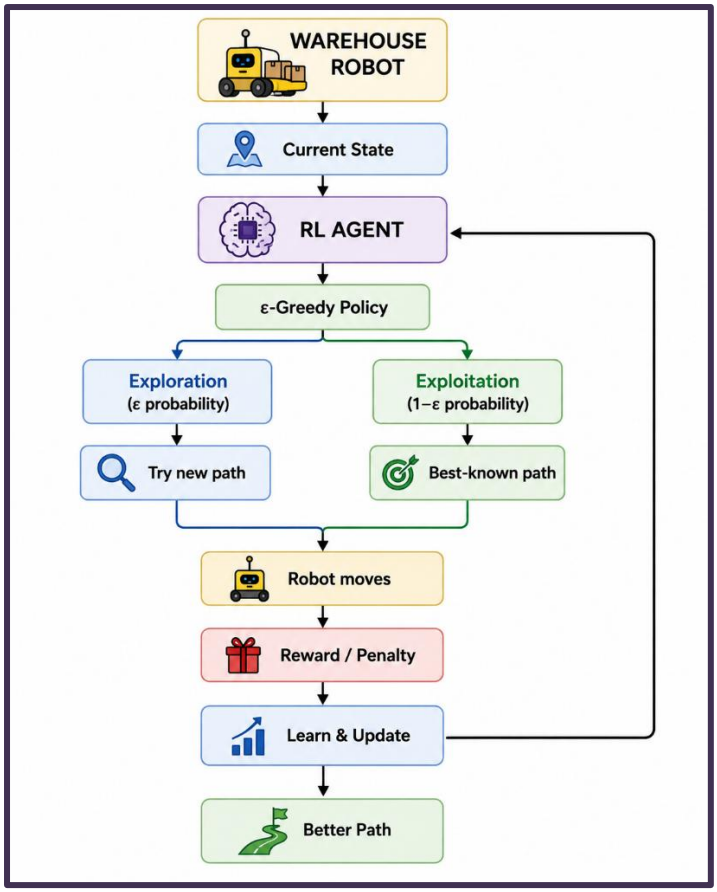


Figure: Exploration–exploitation trade-off in RL-based warehouse navigation

9. Final Answer

The exploration–exploitation trade-off directly affects the warehouse robot's navigation performance. **Insufficient exploration causes the robot to repeatedly exploit familiar but longer paths and prevents it from discovering better routes.** Excessive exploration, however, can cause unnecessary movements and delays. Therefore, an **ε-greedy strategy with ε-decay** is recommended. Initially, the robot explores different routes, and as learning progresses, it gradually exploits the best-known route. Combining this with **reward shaping, experience replay, and collision/step penalties** can improve the robot's ability to find a **short, safe, and efficient path**.

Q3. Reinforcement Learning for Hospital Appointment Scheduling

1. Understanding of the Scenario

The hospital wants to use **Reinforcement Learning (RL)** to schedule patient appointments efficiently.

The main objectives are:

- Reduce **patient waiting time**
- Improve utilization of **doctors, rooms, and staff**
- Handle changing patient arrival patterns
- Reduce appointment cancellations and idle resources
- Improve **patient satisfaction**

This is suitable for RL because appointment scheduling involves **sequential decisions**. A decision made for one patient can affect the availability of doctors, rooms, and time slots for later patients.

2. RL Framework for Appointment Scheduling

RL Component	**Hospital Scheduling**
Agent	RL-based scheduling system
Environment	Hospital appointment system
State	Current bookings, waiting patients, doctor availability, available rooms, time, etc.
Action	Assign patient to a time slot/doctor, reschedule, open or close slots
Reward	Low waiting time, high resource utilization, patient satisfaction
Goal	Learn an optimal scheduling policy

Possible States

The state can include:

- Number of waiting patients
- Current time and date
- Doctor availability
- Room availability
- Number of booked appointments
- Patient arrival rate
- Average waiting time
- No-show rate
- Emergency cases

Possible Actions

The RL agent can:

1. Assign a patient to a time slot.
2. Assign a patient to a doctor.
3. Reschedule an appointment.
4. Open or close an appointment slot.
5. Allocate available rooms.

3. Reward Function

The reward should encourage **short waiting times and efficient resource utilization**.

A possible reward function is:

Where:

- = average patient waiting time
- = idle time of doctors/rooms
- = number of unnecessary rescheduled appointments
- = patient satisfaction
- = weights representing hospital priorities

Therefore:

- **Lower waiting time → higher reward**
- **Lower resource idle time → higher reward**
- **Higher patient satisfaction → higher reward**
- **Unnecessary rescheduling → negative reward**

4. Why RL Is Suitable

A. Sequential Decision Making

Appointment scheduling is not a one-time decision. Each scheduling decision affects future availability.

RL can learn how current decisions influence future waiting times and resource availability.

B. Dynamic Environment

Hospitals are unpredictable.

For example:

- Patients may arrive late.
- Patients may cancel.
- Emergencies may occur.
- Consultation times may vary.
- Doctors may become unavailable.

RL can continuously adapt its scheduling policy to these changes.

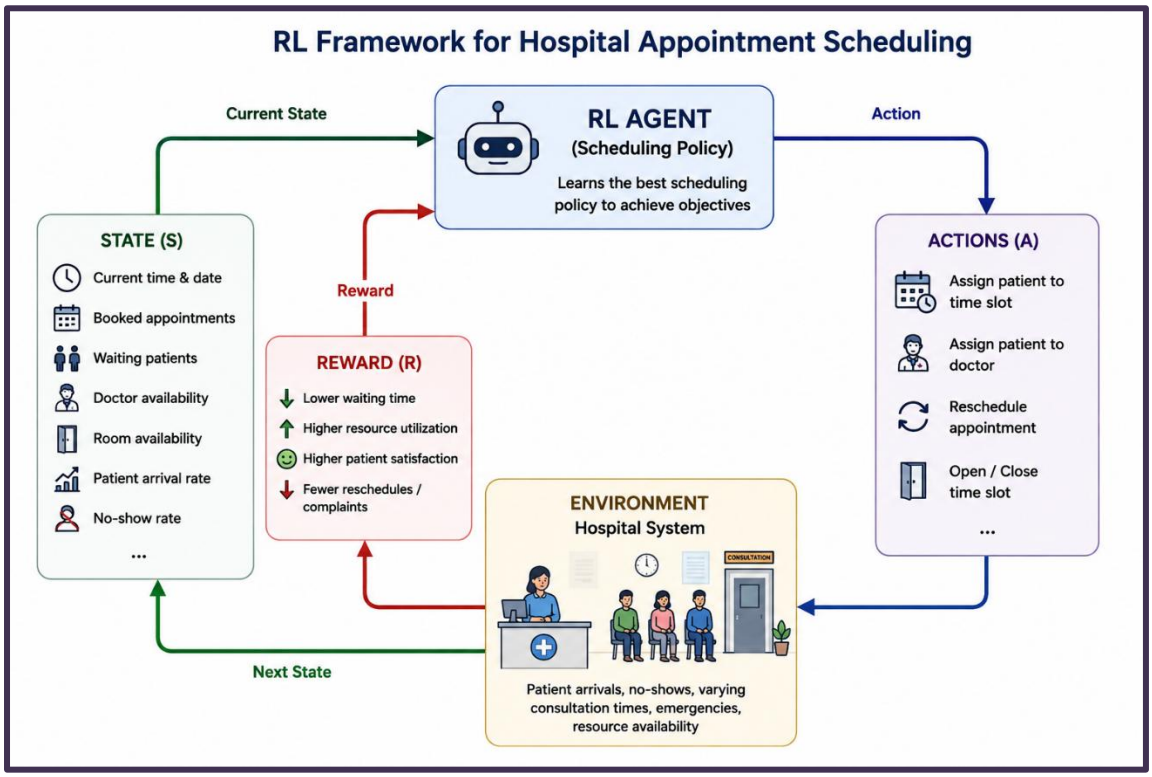
C. Multiple Objectives

RL can optimize several objectives simultaneously:

Waiting time ↓ + Resource utilization ↑ + Patient satisfaction ↑

D. Learning from Experience

The system can learn from previous scheduling outcomes and improve its policy over time.



5. Major Challenge: Delayed Rewards

This is one of the most important challenges in this scenario.

The effect of an appointment decision may **not be immediately visible**.

For example:

Appointment scheduled

↓

Patient waits

↓

Doctor consultation

↓

Other patients are affected

↓

End-of-day waiting time measured

↓

Reward received

A scheduling decision made at **9:00 AM** may affect waiting time and resource utilization several hours later.

Therefore, the agent may find it difficult to determine **which earlier decision caused the final outcome**.

Solution

Use:

- **Discount factor ()**
- **Temporal-Difference (TD) learning**
- **Reward shaping**
- **Multi-step returns**

The discount factor is:

This allows the agent to consider both **immediate and future rewards**.

6. Ethical Considerations

Since this is a healthcare application, ethical issues are extremely important.

A. Fairness

The system should not unfairly prioritize or deprioritize patients based on:

- Age
- Gender
- Income
- Other irrelevant personal characteristics

Scheduling should provide **fair access to healthcare**.

B. Patient Safety

A scheduling decision must never compromise urgent or critical medical cases.

Emergency patients should receive appropriate priority regardless of the optimization objective.

C. Bias

If historical hospital data contains bias, the RL system may learn and reproduce that bias.

Therefore, the training data and policy should be regularly evaluated for fairness.

D. Privacy

Patient information is sensitive.

The system should use:

- Secure data storage
- Access control
- Data minimization/anonymization where appropriate
- Applicable healthcare privacy regulations

E. Explainability

Hospital staff should be able to understand **why an appointment was scheduled or prioritized** rather than blindly following an automated decision.

7. Other Challenges and Solutions

Challenge	**Possible Solution**
Delayed rewards	TD learning, reward shaping, multi-step returns
Patient no-shows	Include no-show probability in the state
Emergency arrivals	Give emergency cases priority constraints
Variable consultation time	Include estimated consultation duration
Large state/action space	Function approximation or Deep RL
Ethical concerns	Fairness constraints and bias monitoring
Safety	Constrained/Safe RL and human oversight
Privacy	Secure and privacy-preserving data handling

8. Decision

RL is suitable for hospital appointment scheduling because it can learn an adaptive scheduling policy in a **dynamic, uncertain and sequential environment**.

However, the system should **not be allowed to optimize only waiting time**. It must consider:

Waiting time + resource utilization + patient satisfaction + fairness + safety

A **constrained RL approach with human oversight** would therefore be more appropriate for real-world healthcare deployment.

Q4. Reinforcement Learning Model for Smart Home Energy Management

1. Understanding of the Scenario

The smart home wants to **reduce electricity consumption** by automatically controlling:

- Lights
- Fans

- Air conditioners

The system observes **user presence and weather conditions** and learns the best control decisions through Reinforcement Learning.

The main objective is:

Minimize electricity consumption while maintaining user comfort.

The RL framework consists of:

Agent → Smart-home control system

Environment → Smart home

State → User, weather and device conditions

Action → Control lights, fans and AC

Reward → Energy saving + user comfort

2. State Space

The **state** represents the current condition of the smart home.

Possible state variables are:

State	Examples
User presence	Present / Absent
Time	Morning / Afternoon / Night
Room temperature	24°C, 28°C, etc.
Outdoor temperature	30°C, 35°C, etc.
Weather	Sunny / Cloudy / Rainy
Light intensity	Low / Medium / High
Device status	Light/Fan/AC ON or OFF
Current energy consumption	Low / Medium / High

Therefore,

where is presence, is time, is room temperature, is outdoor temperature, is weather, is light intensity, is device status, and is energy consumption.

3. Action Space

The **action** represents what the RL agent decides to do.

Possible actions include:

Lights

- Turn ON

- Turn OFF
- Increase/decrease brightness

Fan

- Turn ON
- Turn OFF
- Change speed

Air Conditioner

- Turn ON
- Turn OFF
- Increase/decrease temperature
- Select energy-saving mode

For example:

If the user is absent, the agent may turn OFF unnecessary lights and the AC.

If the user is present and the room is hot, it may turn ON the AC or fan while selecting an energy-efficient setting.

4. Reward Structure

The reward should encourage **energy efficiency without making the user uncomfortable**.

A suitable reward function is:

For example:

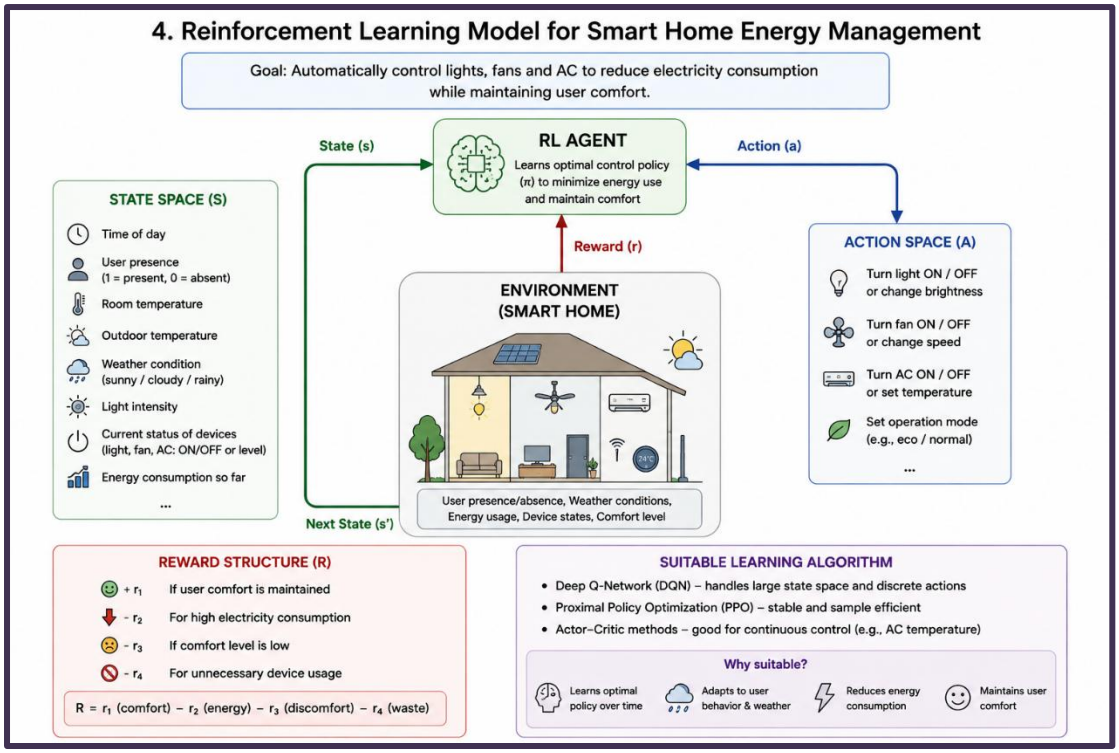
Situation	**Reward**
User comfortable with low energy consumption	**+10**
Significant energy saving	**+5**
High electricity consumption	**−5**
User uncomfortable	**−8**
Device left ON unnecessarily	**−4**

The agent therefore learns that:

Energy saving + user comfort = high reward

while unnecessary electricity usage and discomfort result in negative rewards.

5. RL Model



5. RL Model

It visually shows the **state → agent → action → smart home → reward** cycle without replacing the written explanation.

6. Suitable Learning Algorithm

Deep Q-Network (DQN)

DQN is a suitable choice because the smart home can have a large number of possible states involving:

- User presence
- Temperature
- Weather
- Time
- Multiple device conditions
- Energy consumption

DQN uses a **neural network to approximate Q-values** instead of storing every state-action value in a table.

The basic Q-learning relationship is:

where:

- = current state

- = selected action
- = reward
- = next state
- = learning rate
- = discount factor

The DQN learns which action produces the highest long-term reward.

Why DQN is suitable

1. Handles a **large state space**.
2. Can learn complex relationships between weather, presence and energy usage.
3. Works well with **discrete control actions** such as ON/OFF and different fan speeds.
4. Can continuously improve its control policy from experience.

Alternative: If AC temperature control is treated as a truly continuous action, an **Actor-Critic method such as SAC or PPO** can be considered.

7. Decision Making

The recommended approach is **DQN with an appropriately designed reward function**.

The system should:

1. Observe user presence and environmental conditions.
2. Determine the current state.
3. Select the best device-control action.
4. Observe energy consumption and user comfort.
5. Receive a reward or penalty.
6. Update its policy.
7. Repeat the process.

For example:

User absent + high electricity usage → turn OFF unnecessary devices → receive positive energy-saving reward.

User present + high room temperature → adjust AC/fan → maintain comfort while minimizing energy consumption.

8. Conclusion

Reinforcement Learning is suitable for smart-home energy management because the system involves **continuous decision-making in a changing environment**. By defining an appropriate **state space, action space and reward function**, the agent can learn how to control

lights, fans and AC efficiently. **DQN** is suitable when the actions are discrete and the state space is large. The learned policy aims to **reduce electricity consumption while maintaining user comfort**.

Q5. Reward Design in RL-Based Player Matchmaking

1. Understanding of the Scenario

An online gaming platform uses **Reinforcement Learning (RL)** to match players with opponents of similar skill levels.

The main objectives are:

- Match players with **similar skill levels**
- Maintain fair and competitive games
- Increase player engagement
- Reduce player dissatisfaction and early exits

The RL agent learns matchmaking decisions based on the **rewards received after each match**.

Agent: Matchmaking system

Environment: Online gaming platform

State: Player skill, recent performance, win rate, waiting time, etc.

Action: Select an opponent for a player

Reward: Match quality, fairness and engagement

2. Key Issues

Reward design strongly affects the quality of matchmaking.

Important factors include:

- **Skill difference** between players
- **Match fairness**
- Player waiting time
- Match completion
- Player satisfaction
- Long-term engagement

If the reward focuses on only one factor, the RL agent may learn an **undesirable matchmaking policy**.

For example, maximizing only the number of completed matches may cause the system to ignore skill differences.

3. Reward Definition

A suitable reward can combine several factors:

Where:

- = fairness of the match
- = player engagement
- = skill difference
- = player waiting time
- = weights

Example reward values:

Match outcome	Reward
Very balanced match	+10
Players complete the match	+5
Players are highly engaged	+5
Large skill difference	−8
Player leaves early	−6
Excessive waiting time	−3

This encourages the agent to find matches that are **fair, engaging and reasonably fast**.

4. How Poor Reward Design Causes Bias or Inefficiency

A. Reward Based Only on Winning

If the system gives a high reward when a player wins, it may repeatedly match weaker players against stronger players to produce predictable wins.

Result: Unfair matchmaking.

B. Reward Based Only on Short Waiting Time

The system may match the first available players, even if their skill levels are very different.

Result: Fast but poor-quality matches.

C. Reward Based Only on Engagement

The system might learn to repeatedly match certain players because they play for longer, potentially creating **unequal matchmaking opportunities** for others.

D. Ignoring Player Skill

If skill difference is not included in the reward, the agent has little incentive to create balanced matches.

Result: Inefficient and frustrating gameplay.

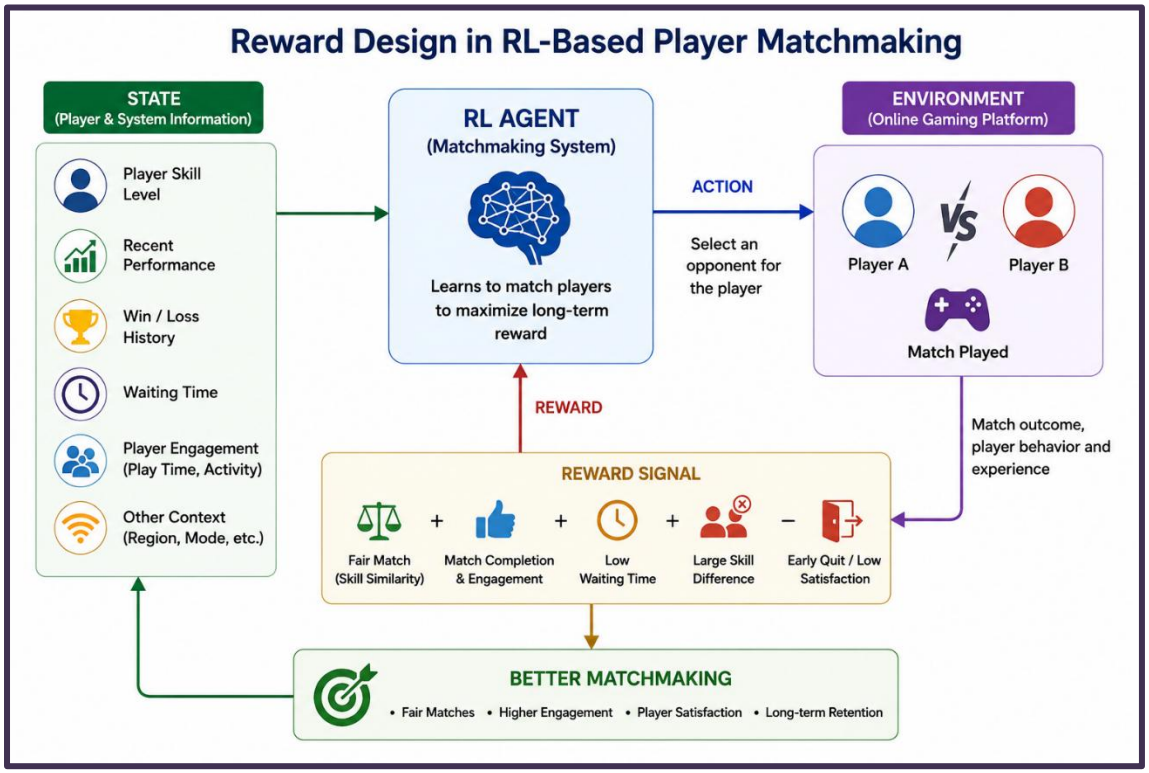
5. Improved Reward Design

A better reward should balance **fairness, engagement and waiting time**.

For example:

This gives greater importance to **fairness** while still considering engagement and waiting time.

The system should also regularly evaluate the reward function to detect whether some player groups are consistently receiving poor matches.



6. Decision Making

The best approach is to use a **multi-objective reward function** rather than optimizing a single metric.

The reward should prioritize:

Fairness → Engagement → Low waiting time

The system should also include **skill difference as an explicit penalty** and monitor matchmaking results for bias.

This produces a more balanced policy instead of one that simply maximizes short-term engagement.

7. Conclusion

Reward design directly determines what the RL matchmaking agent learns. **Poorly designed rewards can produce biased, unfair or inefficient matches**, even if the RL algorithm itself works correctly. A **multi-objective reward function** that considers skill similarity, fairness, engagement and waiting time can produce better-quality matchmaking and improve long-term player satisfaction.

Q6. Reinforcement Learning for Financial Trading

1. Understanding of the Scenario

A financial trading system uses **Reinforcement Learning (RL)** to decide whether to **buy, sell, or hold** an asset based on market conditions.

The objective is to **maximize long-term trading returns while controlling risk**.

Agent: RL trading system

Environment: Financial market

State: Current market and portfolio information

Action: Buy, sell, or hold

Reward: Profit/loss adjusted for risk

2. Definition of States

The **state** represents the information available to the agent before making a trading decision.

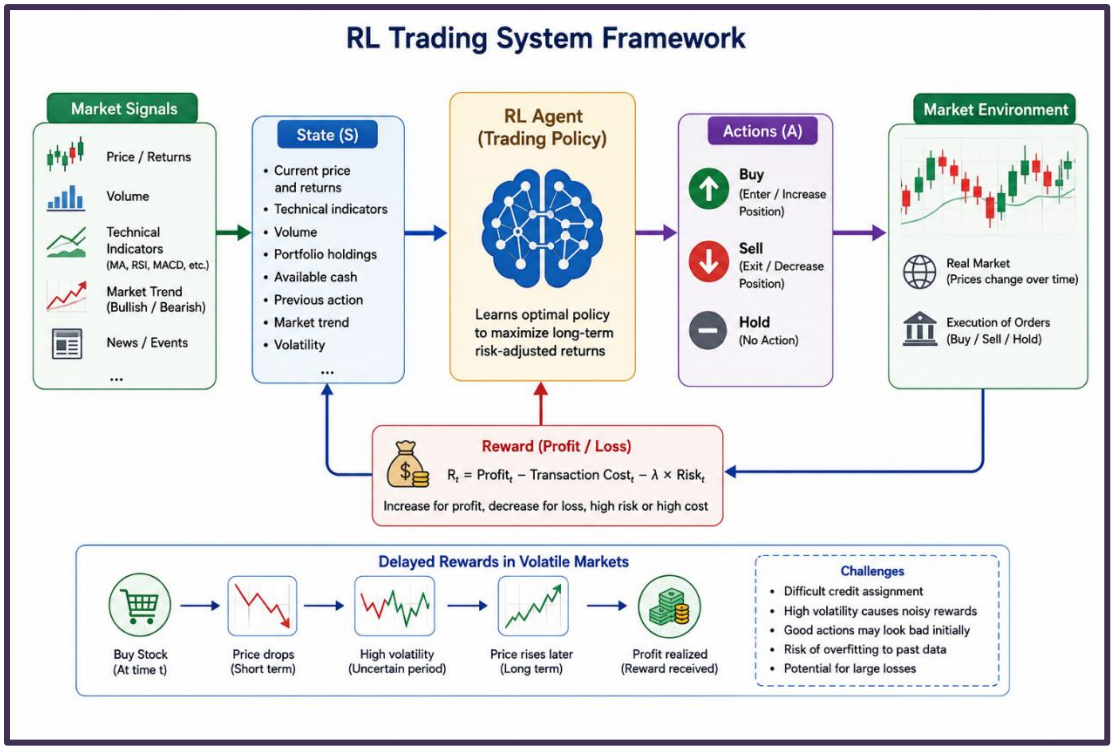
Possible state variables include:

- Current asset price
- Price changes/returns
- Trading volume
- Moving averages
- RSI and other technical indicators
- Market trend
- Current portfolio holdings
- Available cash
- Previous action

For example:

State: Rising price trend + high trading volume + positive momentum + currently holding no shares.

This information helps the agent decide the next action.



3. Definition of Actions

The action space can be:

Action	Meaning
Buy	Purchase the asset
Sell	Sell the held asset
Hold	Take no trading action

For a more advanced system, actions can also represent the **quantity or percentage of capital** to invest.

4. Reward Design

The reward should reflect **profit while also considering trading costs and risk**.

A simple reward is:

where:

- = portfolio value at time
- = portfolio value at the next time step

A more realistic reward can be:

where controls how strongly risk is penalized.

Example

- Profitable trade → **positive reward**

- Loss-making trade → **negative reward**
- Excessive trading → **transaction-cost penalty**
- Very risky position → **additional penalty**

This prevents the agent from focusing only on profit while ignoring risk.

5. Delayed Rewards in Volatile Markets

A major challenge is that the result of a trading decision may become clear **only after several time steps**.

For example:

Buy Stock

↓

Price initially falls

↓

Market remains volatile

↓

Price rises later

↓

Profit is realized

↓

Final Reward

The agent may initially receive a negative or small reward even though the decision eventually becomes profitable.

Risks of Delayed Rewards

1. Difficult credit assignment

It becomes difficult to determine which earlier action was responsible for the final profit or loss.

2. High volatility

Market prices can change rapidly, causing large fluctuations in rewards.

3. Incorrect learning

A good action may appear bad in the short term, while a risky action may temporarily produce a high reward.

4. Risk of overfitting

The agent may learn patterns from historical market behavior that do not work in future market conditions.

5. Large financial losses

Poorly learned policies can make repeated incorrect decisions, resulting in significant losses.

6. Improvements

To reduce these risks, the system can use:

- **Discounted rewards** to consider future returns.
- **Reward shaping** to provide more informative feedback.
- **Risk-adjusted rewards** instead of profit alone.
- **Transaction-cost penalties** to discourage excessive trading.
- **Position limits** to prevent excessive exposure.
- **Extensive simulation and testing** before real-world deployment.

7. Decision Making

A suitable RL design should use a **risk-adjusted reward function** rather than simply maximizing short-term profit.

The agent should consider:

Profit + Future return − Risk − Transaction costs

This is important because a strategy that produces high short-term profit may be dangerous in a volatile market.

Conclusion

RL can effectively learn trading decisions from market states, but **delayed rewards are particularly risky in volatile markets**. They can make it difficult to connect actions with their eventual outcomes and may lead to unstable or risky policies. Therefore, **risk-aware reward design, discounted future returns, transaction-cost penalties and strict risk limits** should be incorporated into the trading system.

Q7. RL Framework for Drone Delivery Route Optimization

1. Understanding of the Scenario

A drone delivery company wants to use **Reinforcement Learning (RL)** to find efficient delivery routes while ensuring that the drone **avoids restricted airspace and obstacles**.

The objective is to:

- Deliver packages on time
- Minimize travel distance and energy consumption

- Avoid restricted areas
- Maintain safe navigation

Agent: Drone/RL controller

Environment: Airspace and delivery area

State: Drone position, battery, weather, obstacles, restricted zones, destination

Action: Move in different directions or hover

Reward: Positive for efficient and safe delivery; negative for unsafe actions

2. State Space

The state represents the drone's current situation.

Possible states include:

- Current drone position
- Destination location
- Battery level
- Wind speed and direction
- Nearby obstacles
- Restricted airspace information
- Distance to destination
- Remaining delivery time

For example:

State: Drone is near the destination, battery is 40%, wind is strong, and a restricted zone is nearby.

This information helps the agent choose a safe route.

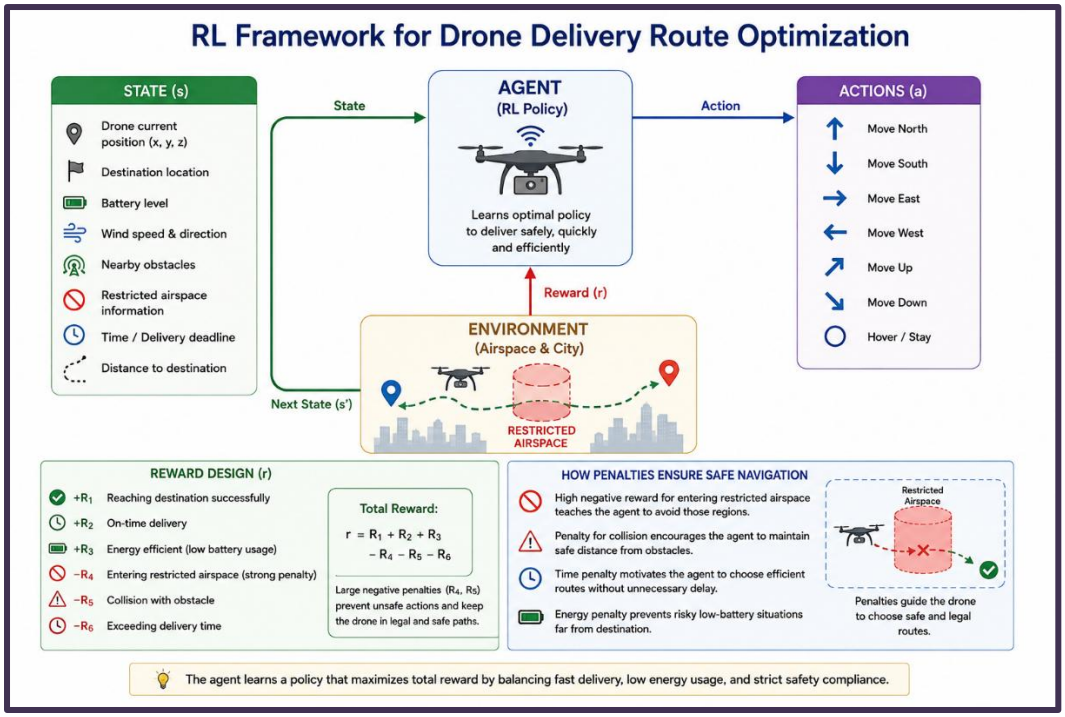
3. Action Space

The drone can perform actions such as:

Action	**Meaning**
Move North/South	Change horizontal position
Move East/West	Change horizontal position
Move Up/Down	Change altitude
Hover	Remain in the current position

The agent selects an action based on the current state.

RL Framework



4. Reward Structure

The reward should encourage **fast, energy-efficient and safe delivery**.

A possible reward function is:

Where:

- = reward for reaching the destination
- = reward for saving energy/distance
- = penalty for entering restricted airspace
- = penalty for hitting obstacles
- = penalty for excessive delivery time

Example:

Situation	**Reward**
Successful delivery	+100
Efficient route	+10
Enter restricted airspace	−100
Collision	−100
Excessive delay	−10

5. How Penalties Ensure Safe Navigation

Penalties are important because they teach the RL agent which actions are dangerous.

- **Restricted airspace penalty:** Makes the agent avoid prohibited zones.
- **Collision penalty:** Encourages maintaining a safe distance from obstacles.
- **Delay penalty:** Encourages efficient routes.
- **Energy penalty:** Prevents unnecessary battery usage.

A **large penalty for entering restricted airspace** ensures that the agent does not choose a shorter route through a prohibited region simply because it saves distance.

Thus, the agent learns:

Safe legal route + efficient delivery = high reward

while:

Restricted zone/collision = large negative reward

6. Decision Making

A suitable approach is **Q-learning/DQN** for discrete movement actions.

The agent repeatedly observes the state, selects an action, receives a reward, and updates its knowledge.

For a real-world drone, the system should also use **hard safety constraints** in addition to RL penalties, because relying only on learned penalties may not guarantee safety.

Conclusion

RL can learn efficient drone delivery routes by considering **position, battery, weather, obstacles and restricted airspace**. Properly designed penalties strongly discourage unsafe actions, allowing the learned policy to balance **delivery speed, energy efficiency and safety**.

Q8. RL for Personalized Online Learning

1. Understanding of the Scenario

A university wants to use **Reinforcement Learning (RL)** to personalize online learning paths according to each student's performance and learning behavior.

The system can dynamically decide **which topic, difficulty level, or learning activity** should be provided next.

Agent: RL-based learning system

Environment: Online learning platform

State: Student performance, progress, quiz scores, engagement, etc.

Action: Select the next learning content/activity

Reward: Improved learning and meaningful engagement

2. Key Issues

Important factors affecting the system are:

- Student engagement
- Quiz/test performance
- Learning progress
- Time spent on content
- Difficulty level
- Completion rate
- Individual learning pace

The main challenge is designing rewards so that the system does not focus only on **short-term engagement** while ignoring actual learning.

3. Reward Design

A suitable reward function can combine learning and engagement:

Where:

- = learning improvement/performance
- = meaningful engagement
- = content completion
- = excessive difficulty or frustration
- = weights

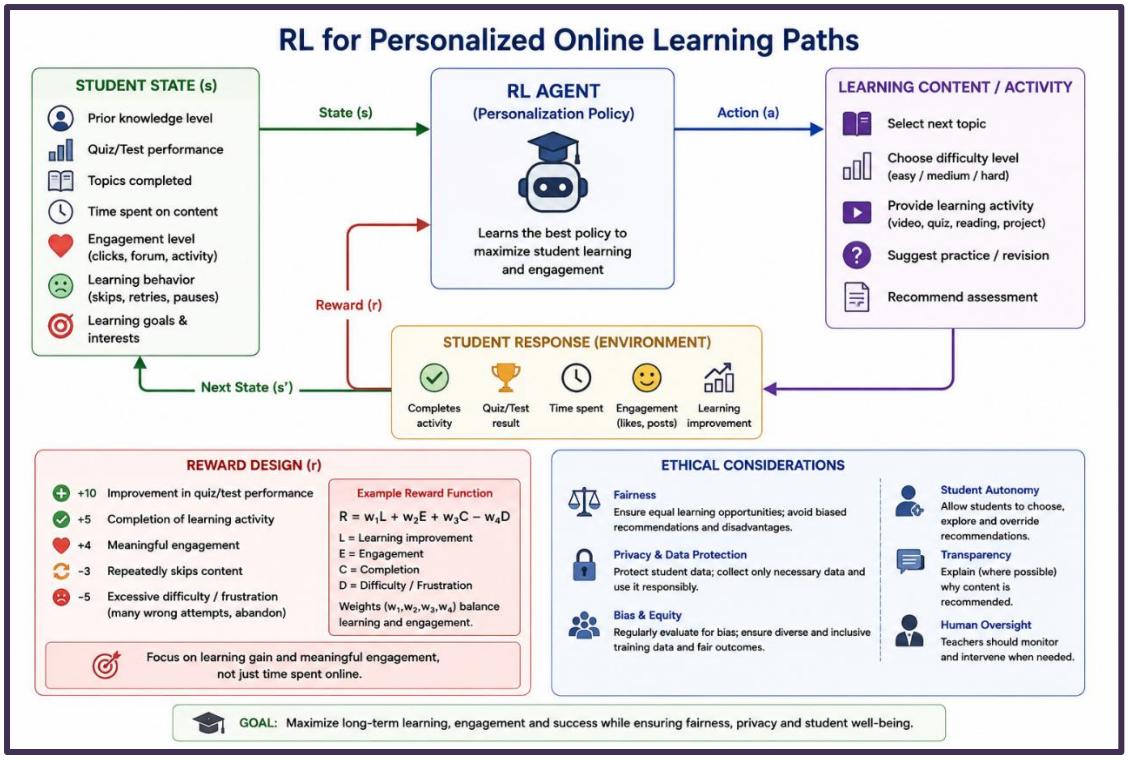
Example:

Student response	**Reward**
Improved quiz performance	+10
Completes learning activity	+5
Active and meaningful participation	+4
Repeatedly skips content	−3
Excessive difficulty/frustration	−5

Effect of Reward Design

If the system rewards **only time spent online**, it may recommend easy or entertaining content simply to keep students online.

If the reward emphasizes **learning improvement and engagement**, the system is more likely to recommend content that actually helps the student progress.



4. Ethical Considerations

Dynamic adaptation can improve learning, but it must be used responsibly.

A. Fairness

The system should provide equal learning opportunities and should not disadvantage students based on irrelevant personal characteristics.

B. Privacy

Student performance, behavior and learning history are sensitive educational data. It should be securely stored and used only for appropriate purposes.

C. Bias

If the training data contains bias, the RL system may repeatedly recommend easier or fewer opportunities to certain groups of students.

Regular fairness evaluation is therefore necessary.

D. Student Autonomy

Students should not be completely controlled by the algorithm. They should have the ability to **choose topics, change learning paths, or override recommendations**.

E. Transparency

Students and teachers should understand that content is being personalized and, where practical, why particular content is recommended.

5. Decision Making

The best approach is a **multi-objective reward function** that prioritizes:

Learning improvement + meaningful engagement + progress

rather than simply maximizing screen time.

The university should also use **fairness checks, privacy protection and human/teacher oversight** to ensure that dynamic personalization benefits students without creating discrimination or excessive dependence on the algorithm.

Conclusion

RL can create personalized learning paths by continuously learning from student performance and engagement. However, **reward design determines what the system learns to optimize**. A carefully designed reward that balances learning and engagement, together with **fairness, privacy, transparency and student control**, can make dynamic learning personalization both effective and ethical.

Q9. RL-Based Traffic Light Control

1. Understanding of the Scenario

A traffic authority uses **Reinforcement Learning (RL)** to control traffic signals at busy intersections. The objective is to **reduce congestion and waiting time while maintaining emergency vehicle priority and pedestrian safety**.

Agent: RL traffic controller

Environment: Road intersection

State: Traffic queues, waiting times, signal phase, pedestrians, emergency vehicles

Action: Change or extend traffic-light phases

Reward: Reduced congestion while maintaining safety

2. Key Issues

The RL system must balance:

- Vehicle queue length
- Vehicle waiting time
- Traffic flow
- **Emergency vehicle priority**
- **Pedestrian crossing safety**
- Signal timing

The main challenge is that **congestion reduction must never override safety requirements**.

3. RL Model and Constraints

State Space

The state can include:

- Number of vehicles in each lane
- Average waiting time
- Current traffic-light phase
- Number of waiting pedestrians
- Emergency vehicle presence and distance
- Average traffic speed

Action Space

The agent can:

- Keep the current signal
- Switch to another phase
- Extend/reduce green time

Reward Function

A suitable reward can be:

Where:

- = vehicle queue length
- = vehicle waiting time
- = traffic flow
- = pedestrian delay/safety violation
- = emergency vehicle delay

Incorporating Constraints

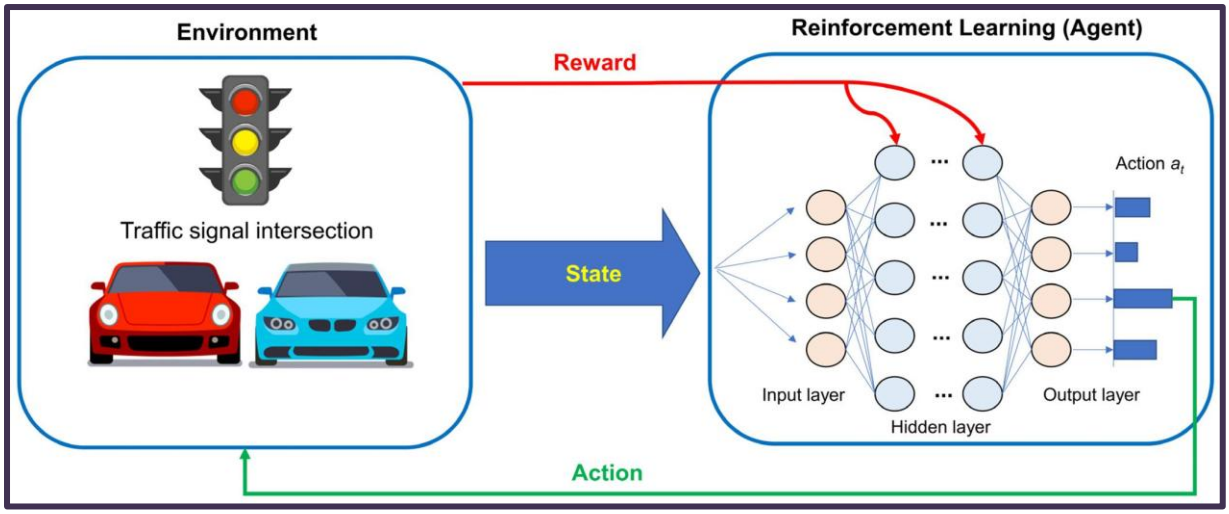
Emergency vehicle priority:

If an emergency vehicle is detected, the controller can apply a **rule-based override** or large penalty for delaying it.

Pedestrian safety:

The system must ensure minimum crossing time. A large penalty can be given if the signal changes before the safe crossing period is completed.

These can be treated as **hard safety constraints**, rather than allowing the RL agent to trade safety for a higher traffic-flow reward.



4. Impact on Congestion Reduction

With proper learning, the RL controller can:

- Reduce vehicle queue lengths
- Reduce average waiting time
- Improve traffic flow
- Reduce unnecessary idling
- Adapt signal timing to changing traffic conditions

For example, during peak traffic, the agent can give **more green time to heavily congested lanes**, while still respecting pedestrian and emergency constraints.

5. Decision Making

A **constrained RL approach** is most suitable because the system must optimize traffic flow **within safety limits**.

The decision process is:

Observe traffic → Check safety constraints → Select signal action → Receive reward → Update policy

Emergency vehicle priority and pedestrian safety should have **higher priority than congestion reduction**.

Conclusion

RL can significantly reduce congestion by dynamically adjusting traffic signals according to real-time traffic conditions. However, **safety constraints must be explicitly incorporated** so

that the system never sacrifices emergency response or pedestrian safety merely to improve traffic flow.

Q10. Traditional Optimization vs. Constrained RL for Machine Scheduling

1. Understanding of the Scenario

A manufacturing plant needs to schedule multiple machines while resources such as **labor, power, raw materials, and machine capacity are limited**.

The objective is to:

- Complete more jobs
- Reduce production delays
- Minimize operational cost
- Efficiently use available resources

The main challenge is that production conditions can change due to **new orders, machine breakdowns, changing priorities, or resource availability**.

2. Key Issues

The scheduling system must handle:

- Limited machine and resource capacity
- Different job priorities and deadlines
- Machine breakdowns
- Changing production requirements
- Resource conflicts
- Need for quick rescheduling

Traditional methods may require **re-optimization whenever conditions change**, while RL can learn a policy that adapts to changing conditions.

3. Traditional Optimization vs. Constrained RL

Aspect	Traditional Optimization	Constrained RL
Approach	Calculates an optimal schedule using a mathematical model	Learns a scheduling policy through experience
Examples	Linear Programming, Integer Programming, Constraint Programming	Constrained RL, Safe RL

Dynamic changes	Usually requires re-optimization	Can adapt to changes
Resource constraints	Explicitly included in the model	Enforced through constraints/penalties
Speed after training	May require repeated optimization	Can make decisions quickly
Adaptability	Lower in highly dynamic environments	Higher

Traditional optimization is effective when the environment is **stable and well-defined**.

Constrained RL is more suitable when production conditions are **uncertain and continuously changing**.

4. Constrained RL Framework

State

The state can include:

- Current machine status
- Job queue
- Processing time
- Resource availability
- Job deadlines
- Current production progress

Actions

The RL agent can:

- Assign a job to a machine
- Change job sequence
- Delay or hold a job
- Change job priority

Reward

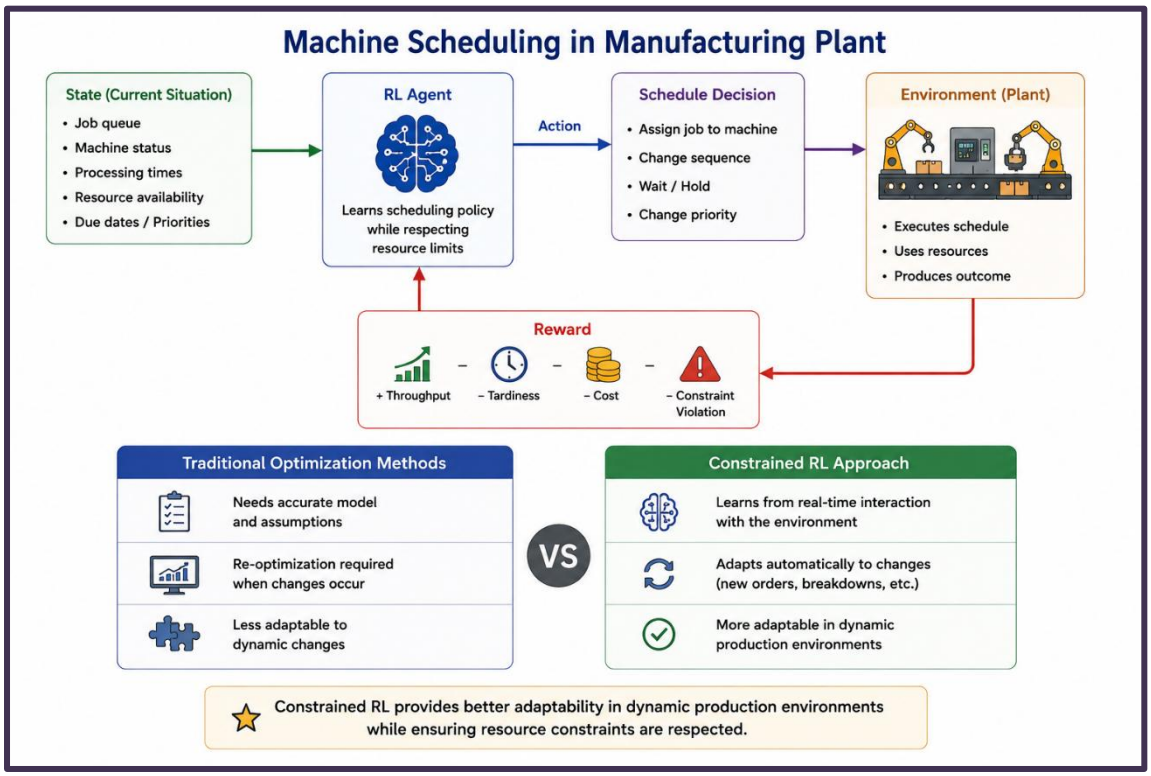
A suitable reward can be:

Where:

- = production throughput
- = delay/tardiness
- = operational cost

- = resource constraint violation penalty

The agent aims to **increase throughput while reducing delay, cost, and constraint violations**.



5. How Constraints Are Handled

In constrained RL, the agent must respect limitations such as:

- Maximum machine capacity
- Available labor
- Power limits
- Raw material availability
- Production deadlines

Unsafe or infeasible actions can be **blocked using action masking** or assigned a **large negative penalty**.

For example:

If a machine has reached its capacity, the agent should not assign another job to it.

This prevents the learned policy from producing infeasible schedules.

6. Decision Making

Constrained RL provides better adaptability for a highly dynamic manufacturing environment.

Traditional optimization is preferable when:

- The production environment is stable.
- Changes are infrequent.
- An accurate mathematical model is available.

Constrained RL is preferable when:

- Orders change frequently.
- Machines may break down.
- Resources vary over time.
- Real-time scheduling decisions are required.

Therefore, for a **dynamic production environment**, constrained RL is the better choice because it can learn from experience and adapt its scheduling decisions without completely solving the optimization problem again each time.

7. Conclusion

Traditional optimization can provide highly accurate schedules for **stable production environments**, but it may become slow when frequent changes require re-optimization. **Constrained RL offers better adaptability** because it learns a policy that can respond to changing production conditions while respecting resource limitations. Hence, constrained RL is more suitable for **dynamic and complex manufacturing environments**.
